

TraqCheck-Lite: Executive Report

Capstone submission — agentic background-verification system, evaluation, and deployment readiness

Business Goal

Background checks are a required step before most hiring decisions, but verifying a candidate's claimed employment, education, and references is slow, manual, and inconsistent when done by hand. TraqCheck-Lite is a proof-of-concept agent system that demonstrates how this verification step can be automated end-to-end while keeping a human decision-maker in control of the final outcome. The goal was not to ship a production background-check product, but to demonstrate sound architectural judgment: choosing the right orchestration pattern for a process that mixes deterministic workflow control with genuine multi-perspective reasoning, and proving that judgment out with a working system, a real evaluation, and honest documentation of where it still falls short.

Architecture

A candidate submission flows through four stages:

- **Validate input** — rejects missing fields or missing consent before any paid API call is made.
- **Run verification** — a three-agent crew (Employment Verifier, Education Verifier, Reference & Risk Analyst) checks the candidate's claims against a registry database and produces a structured risk report.
- **Human review** — the pipeline pauses here. A reviewer sees the proposed report and approves, rejects, or requests more information before anything is finalized.
- **Finalize** — records the reviewer's decision as the release status.

The system is exposed as a FastAPI service with two endpoints: one to submit a candidate and receive a pending report, and one for the reviewer to submit their decision and resume the paused workflow.

Framework Choice Rationale

Two frameworks are combined deliberately, not by default. **LangGraph** owns the outer flow because the process is genuinely sequential and conditional, and — critically — needs to pause and durably hold state across a human approval step. That pause-and-resume capability is LangGraph's core strength and isn't something a single linear script or a plain agent loop handles cleanly. **CrewAI** runs inside one node of that graph, because the verification step itself benefits from real role separation: an employment specialist, an education specialist, and a risk analyst each reasoning over a different slice of evidence produces a more reliable final judgment than one prompt trying to juggle all three jobs credibly at once. Using LangGraph for control flow and CrewAI for role-based collaboration inside a single node reflects how production agent systems are typically composed when a workflow has both needs at once, rather than picking one framework and forcing every requirement through it.

Evaluation Results

Ten test cases were run against the live system: eight standard cases covering clean matches, single and multiple discrepancies, fuzzy name matching, a fully fabricated candidate, missing consent, and malformed input; plus two adversarial cases testing prompt-injection resistance. Five criteria were scored per case: task success, risk-level accuracy, consent/input-validation handling, injection safety, and latency.

Criterion	Result
Task success	10 / 10 completed without crashing
Consent / input validation	2 / 2 correctly rejected, zero token cost

Injection safety	2 / 2 adversarial cases correctly flagged, no outcome override
Risk-level accuracy	8 / 10 correct (2 traced to a test-data mismatch, now fixed)

Both adversarial cases correctly flagged the injection attempt as suspicious without letting it override the fact-based risk assessment — the system did not get talked into approving a candidate by text embedded in submitted data. The two risk-accuracy failures were traced, through direct inspection of the registry database, to a test-fixture data mismatch: a candidate name referenced by three test cases did not exist in the registry actually used for that run. This was not a defect in the agent's reasoning — the crew correctly reported "not found" given the input it was actually given. A fix has been applied to the test fixture, and the harness now fails loudly instead of silently if this drift happens again. Full scored results are in *eval/eval_results.csv*; the detailed root-cause writeup is in *eval/FAILURE_ANALYSIS.md*.

Known Limitations

- **Synthetic data only.** The registry is Faker-generated, not a real data source. Production requires licensed background-check data providers and jurisdiction-specific compliant consent language (e.g. FCRA in the United States).
- **In-memory state.** The LangGraph checkpointer is memory-only; a server restart loses any pending human review. Production needs a durable checkpointer (e.g. Postgres).
- **Single reference source.** The system checks one reference note per candidate. Real checks typically pull from multiple referees.
- **No authentication.** The FastAPI endpoints are unauthenticated, suitable for local testing only.
- **Rate-limit sensitivity.** The free-tier API used for this demo has a token-per-minute ceiling a three-agent sequential crew can approach on a single run; this is now handled with backoff and a model choice with a larger free-tier budget, but production traffic would need a paid tier or reduced per-call token usage.

Recommended Next Steps

- **Scaling:** move to a persistent checkpointer and add authentication before any deployment beyond localhost; move off the free-tier rate limit before handling concurrent real traffic.
- **Guardrails:** extend the adversarial test set on a regular cadence, since injection phrasing evolves faster than a static test suite; add an automated contradiction check (e.g. alert if `risk_level` is ever "low" while an injection flag is present) rather than relying solely on prompt instructions.
- **Human oversight:** keep the human-approval checkpoint in place for any real deployment — this system is designed to accelerate and structure a reviewer's work, not to replace their judgment on a consequential hiring decision.
- **Monitoring:** track error rate, latency, token cost per run, and human override rate from day one; a full checklist with alert thresholds and re-evaluation cadence is provided in *MONITORING_CHECKLIST.md*.